

Scrum in a Software Engineering Course: An In-Depth Praxis Report

Andreas Scharf
Software Engineering Research Group
University of Kassel
Kassel, Germany
andreas.scharf@cs.uni-kassel.de

Andreas Koch
Software Engineering Research Group
University of Kassel
Kassel, Germany
andreas.koch@cs.uni-kassel.de

Abstract

Working in a team for a limited period of time and to deliver a high quality product are some of the skills students should gain during their studies at the university. Therefore universities should offer software engineering courses teaching agile methodologies broadly used in the industry as well. These courses need to simulate a real world working environment introducing agile methods like XP or Scrum. We present the design of our Scrum based undergraduate course Software Engineering 1, provide in-depth information about important course details and also give recommendations for tools we find useful. Unlike other software engineering courses we have a large staff of up to 11 supervisors allowing us to simulate close to real world working conditions. We provide every team with a tutor who is responsible for observing his team and answer questions. Additional research assistants are playing the role of customers.

1. Introduction

As most other courses of this type [2, 5], the main goal of our course *Software Engineering 1* (SE1) is to teach students how to deliver a high quality piece of software within a limited amount of time working in a team. The key qualifications needed to undertake such a project are requested by industrial companies as well and it is important to get this knowledge across to the students [3]. SE1 is a mandatory course intended for undergraduate students in the 4th term and takes place once a year in the summer term.

We usually have 50-70 students who already collected some background knowledge in lectures like “Introduction into programming”, “Programming Methodologies”, “Introduction to C” and “Algorithms and Data Structures”. Although all students have heard these lectures, their skills vary dramatically as we will explain later.

During 20 weeks, 6-8 groups of 7-8 students have the chance to work on a (for students) large project. Providing a self-written game server we require the teams to develop a game client, a map editor and an artificial intelligence. SE1 is the first (and usually the last) course where students really have to work together in a small team. This raises some interesting (while partly expected) situations, opinions and results which we will cover in detail in the following sections.

We decided to use Scrum [9] as project management method for several reasons: First of all Scrum is the most used agile methodology in industry [10]. Our department feels constrained to not teach students content that might never be needed again, but instead we try to convey as much praxis relevant knowledge as possible to our students. Showing them one of the most popular project

management methods belongs to this praxis. Second, we found that the Scrum process fits very nicely into our university time frame of one term with 16 weeks. As explained later, we split 16 weeks into 4 releases with 2 sprints in each release. This is different from other courses [6] using Scrum which are not using releases and only have 2-3 sprints that last 2 weeks each. 4 more weeks are required before the first sprint actually starts.

One of the main differences to related reports is that we try to design the complete course as realistic as possible. We have customers (played by two of our research assistants) who want to have a functional product with specific needs which may vary from team to team. We have presentations at the end of each release which are held by the Scrum Master and Product Owner of each team. Communication between the customers and the teams is not possible at every point in time which greatly simulates real world situations.

This paper is structured as follows: Section 2 gives a brief overview of related work. In section 3 we describe what we want the students to learn and why. Section 4 describes the course content of SE1 in summer 2012 along with the course structure, used tools and all other related things. We then present a short evaluation and conclusion in section 5.

2. Related Work

Most universities in the field of computer science have special courses for teaching software engineering methodologies. However, to really understand and assimilate agile methodologies like Extreme Programming or Scrum students have to do it. It's not possible to run a project using Scrum by only reading a book or listening to someone who is teaching such a methodology. Hence most courses teach those techniques by requiring the students to complete a given project in a team during a given amount of time [5–8]. Because we decided to use Scrum in our course for the last four years, a closer look to related work also teaching Scrum in software engineering courses is nearby. Mahnic [5] provides detailed information about their software engineering capstone course at the University of Ljubljana which is very similar to our own course at the first look. They also use Scrum and students are working on a project for 15 weeks. One of the main differences is related to the Scrum roles Product Owner and Scrum Master. Both roles are held by the teacher himself instead of letting the students play these roles. Of course, this has the advantage of having fine grained control over stories, tasks and other artifacts related to Scrum. The teacher also plays the role of the all-knowing customer which is available all the time. In reality this usually does not hold true. Whereas this is really convenient for the teams, we think that this is a critical point which has to be simulated as realistic as possible. Therefore we have dedicated research assistants playing the role of customers and teams have to make an appointment to talk to them. Additionally, the Scrum roles Product Owner and Scrum Master are up to the team. Another major difference is the way Mahnic describes the creation of the product backlog: Because the teacher is the Product Owner, he also created an initial product backlog already containing 60 user stories. Students then only had to prioritize these stories instead of creating the stories themselves. However, in our opinion this step in the Scrum process is quite important and students should learn how to correctly formulate such stories.

3. Learning Objectives

The most courses during the studies at a university are targeted on grading each student individually. This means team work is mostly not encouraged or even prohibited. Regardless of whether the course teaches primarily theoretical or practical knowledge, self-organized study groups are

mostly the only experience of team work. Therefore one of our main learning objectives for the students of this course is experiencing team work in all facets. This includes working together with unknown students and reaching individual goals (e.g. a good grade) with the help of and by helping team members. To achieve this goal we let the students work on a realistic software project. While the necessary programming takes the most of the time during the 20 weeks of this course, it is only secondary with regard to the learning objectives.

Although this course is scheduled in the 4th term, the previous knowledge of the participating students is very heterogeneous. Some have already finished an apprenticeship in the IT industry; other students only have a basic programming knowledge from their first three terms. Because of this different prior knowledge, the learning objectives for each individual student vary. While the experienced programmers are encouraged to share their knowledge with the less skilled team members (e.g. with pair programming), these weaker students have the opportunity to improve their technical skills during the course.

Another learning objective of this course is the improvement of each student's presentation skills. This includes the elaborate preparation of such a presentation. In this context the students shall get a glimpse about the communication and negotiation with a customer. This results in a wide spread of tasks, e.g. how and what to communicate with a customer or how to prepare a meeting. So they have to learn, that they need to fulfill the customers' demands or provide a good argumentation why something is/was not possible.

To fulfill our requirements concerning the amount of required code (1000 LOC) it is important that all team members establish a common data model and communicate their design and knowledge at an early point in time. Without such a big picture of the overall system we experienced that it is not possible to implement even a small part of functionality in an appropriate time because most work is spent on crawling and understanding code written by other team members.

Apart from the programming skills, most students participating in this course are very inexperienced in the usage of tools for software engineering. This includes IDEs like Eclipse¹ or version control systems like Subversion. Therefore those tools are demonstrated and explained in the lecture at the beginning of the course and the staff is always available if any technical problems occur.

The learning objectives of course have a huge impact in our grading policy as described in section 4.3. This grading policy is revealed to the students during the first lecture weeks.

4. Approach

In this chapter we will describe the structure and design of SE1 from summer term 2012. Since this course is offered once a year, several improvements were made in comparison to past courses and we will discuss the most important ones at appropriate points throughout this chapter.

To support all participating students as best as possible, our staff is made up of 11 people: The professor, two research assistants supervising the course in general, two research assistants playing the role of customers and six student research assistants for supervising each team as their tutor. Research assistants are doing their PhD whereas student research assistants are undergraduate students. Both are employed at our research group, completed SE1 and have a lot of experience regarding this course. Not only is this a lot of people, the mentoring is very time consuming and exhausting for the whole staff. The reason is what we call "managed freedom", which basically means to give as much support as required but not more than needed. Every type of staff member has to invest a different amount of time at different points within the course. This fact is visualized in Figure 1.

¹<http://www.eclipse.org>

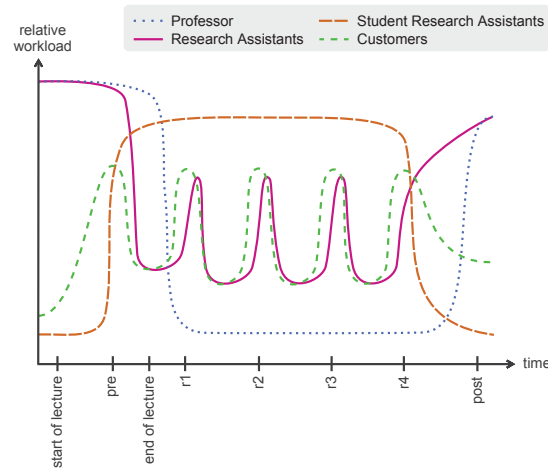


Figure 1. Planned workload of involved staff members

The professor begins the course with a series of lectures covering basic concepts and tools utilized in the course. Two research assistants have to prepare and maintain the game server, supervise the student research assistants and solve upcoming administrative problems. Research assistants playing the role of customers have to hold meetings with each team before and after each release. The six student research assistants have to support their team several hours a week, for example by answering technical questions. While it is important to support and guide the students especially in the first four weeks as best as possible on the one hand, the team tutors have to take care not to overact while helping their team on the other hand. As we will describe later it is difficult to provide consistent help across all different teams and their tutors.

As already mentioned, we use Scrum for project management. We made best experience with four releases with two sprints in each release. Each sprint lasts two weeks which sums up to 16 weeks of work on the project. Each team has to present their results after each release and a final tournament will be held after the end of the fourth release. The exact duration of all mentioned blocks as well as important deadlines are shown in Figure 2. The customer release meetings are always held before the upcoming release (release 1 is the exception). This way we assure that the tasks can be planned before the first sprint of a release is scheduled. It is worth mentioning that a large amount of work has to be invested into server- and client components which are developed by the research assistants before the course actually starts.

During the last years we used different game projects as the base for SE1. Although some students complained about that type of project, the majority was very motivated to write their first multiplayer game. We provide a self-written game server and the teams are requested to write a game client, a map editor and an artificial intelligence. All used technologies to implement these functionalities are up to teams but we encourage them to use technologies our tutors are familiar with. Using this kind of project type enables us to define a lot of different self-contained tasks which allows teams to distribute the work among team members. We also give further assistance and recommendations of how to split-up these big tasks into smaller Scrum-like requirements, user stories and tasks.

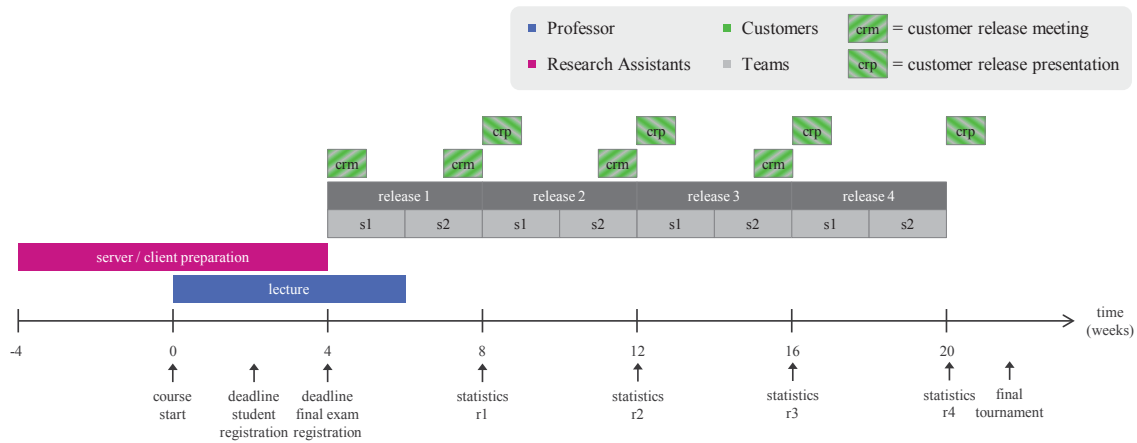


Figure 2. Course overview showing durations of working units and important deadlines

4.1. Preparation

In our opinion one important aspect of putting together a team is randomness. Several years ago we let the students choose their teams themselves which led to happy students on the one hand but to some failing teams on the other hand. The reason is that after the “hacker” teams have emerged, there always were teams consisting of weaker students only. Latter teams were often subject to fail. We are therefore using a mixed form of building teams: Students have to register online within the first week of the course (as shown in Figure 2) where they can specify one other student who will be in the same team. If this other student also specifies the first one, they make it up into the same team. After all students have registered themselves we mix up three to four pairs to one team. Using this technique combines real-life working conditions (working with new and probably unknown people) with the “good feeling” of knowing at least one member within the team.

After team creation is completed, each team gets their own tutor in the form of a student research assistant. This tutor is responsible for kick-starting the team in the first days to allow for a smooth start. He is also the first-point-of-contact if the team has any problems they cannot overcome themselves and supervises the team during weekly mandatory team meetings. Especially the administrative roles of Scrum Master and Product Owner need to be supervised to assure the teams are following the Scrum method. To have a tutor for each team of course requires a large staff but we made the experience that this is the only way to provide enough support. We observed that the teams usually need a great amount of assistance especially in the first two weeks. The reason is that they have to work with a bunch of new software and most of them do not have any experience with project management methods like Scrum and tools like Agilo [1]. Because all teams are working in parallel it is not possible to only have three to four tutors and support the teams sequentially. Besides that, the tutor’s observations are part of our grading system described later.

For the teams to coordinate themselves and also make use of version control facilities it seems likely to use one of the publicly available project management tools. In our special case we had the requirement to be able to self-host the tool which narrows the number of usable tools. We evaluated several Scrum tools and adopted Kunagi² [4] in winter term 2011/2012 and Agilo in summer term

²We used version 0.21

2011 and 2012. We prefer Agilo because it can be installed as a Trac plugin and thus provides source code management facilities out of the box. Furthermore, the Scrum workflow for planning requirements, stories, sprints and tasks feels more natural in comparison to Kunagi. In addition, we experienced several major bugs during the usage of Kunagi.

Using a dedicated virtual machine we provide an Agilo instance for each team. At the first team meeting the tutor initializes the Agilo instance by creating an account for each team member and assigning an appropriate Scrum role for everyone.

As already mentioned, two research assistants play the role of customers. They introduce themselves in one of the first lectures (usually the second or third) and present the rough requirements as well as some of their administrative tools like their own blog³ and a Google calendar. These tools must be used by the teams to reserve dates for requirement discussions and release presentations. To raise the level of realism, customers wear a suite and also have a fictitious company. At this point we already thought of involving some “real” customers which would make the course even more realistic. The problem here is to find a company providing some men power at no charge. However, we find that having some dedicated customers (even if they are represented by research assistants) instead of communicating all necessary information through the tutors for example is better for several reasons:

- Limiting the amount of communication between teams and customers simulates the “real world” more adequate. The teams have to collect questions and ask them all at once during a pre-agreed meeting with the customer.
- The teams usually hold the customers in high esteem. This in turn leads to a better preparation concerning questions and presentations. Having “real” customers would increase this effect.
- Splitting up responsibilities for helping students in the tutor role on the one hand and grading them as a customer on the other hand greatly avoids role conflicts.

The last tool which we are using is a dedicated Google group maintained by the research assistants. Teams can ask any kind of technical question concerning the provided game server or report bugs. We introduced this concept because we got many mails from different teams often asking the same questions. Providing a common area for such questions not only solves the problem of answering the same question twice but also led to discussions across teams which is a behavior we greatly appreciate.

4.2. During the Releases

To simulate the course as realistic as possible, we try to provide strict deadlines the students have to follow. Each release follows the same cycle as shown in Figure 2: The week before each release cycle starts the release meetings for each team are scheduled. In this meeting the team’s Product Owner and the customer analyze all release related requirements. In the years before we let the students decide how to prepare this meeting based only on a roughly outlined release plan. This resulted in qualitatively quite varying meetings. To improve the quality in the current course, we decided to publish a requirement catalog one week before the start of the next release cycle on the customer’s blog. Additionally we explicitly asked the Product Owner to prepare this meeting with mock ups, design ideas and/or possible problems.

³<http://wasp-enterprises.de/>

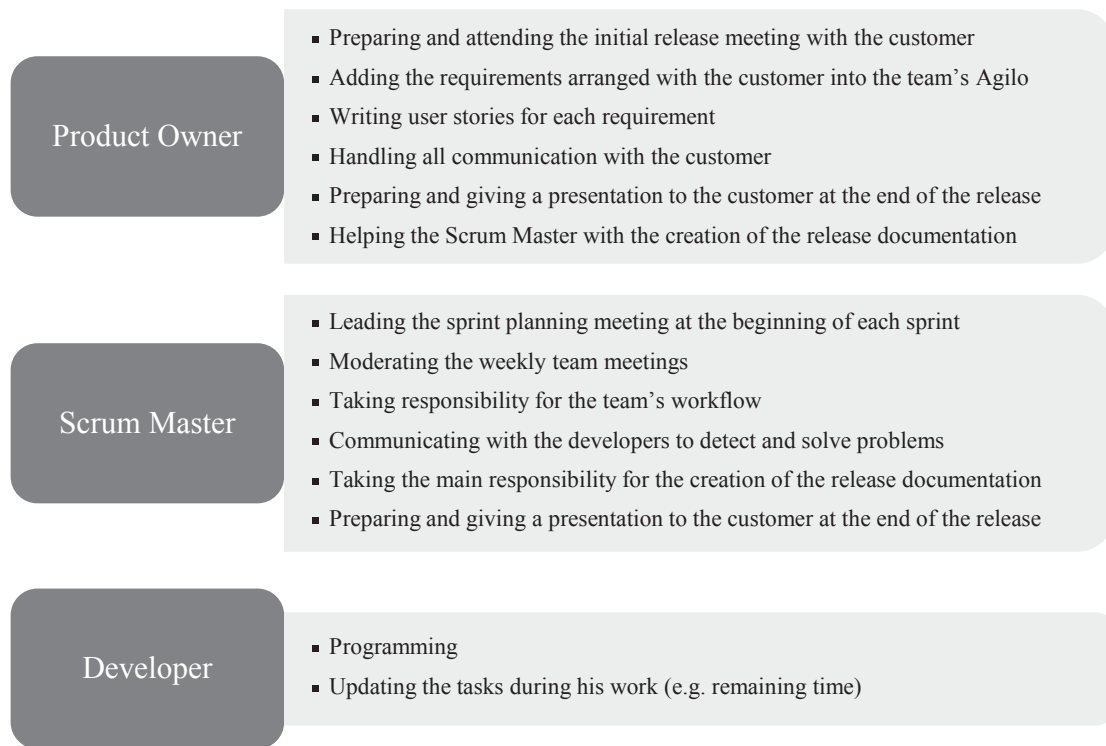


Figure 3. Tasks for each Scrum role

After the release meeting a sprint planning meeting with the whole team has to be arranged as soon as possible which has to follow the rules as Scrum describes it. This means in preparation for this meeting the Product Owner has to create requirements and related user stories in the team's Agilo instance following the customers' needs. In addition these artifacts need to be assigned to the two sprints of this release. Based on this work the Product Owner presents the goals of the upcoming release at the first part of the sprint planning meeting to the rest of the team. The user stories are now discussed by the Scrum Master and the developers and appropriate tickets for the upcoming release are created. Each ticket requires story points (for prioritization) and an initial time estimation to provide a valid data basis for the burn down charts. After the sprint planning meeting the developers may start with their tasks.

At the end of every release cycle a presentation by the Product Owner and Scrum Master has to be scheduled. The years ago we recognized that most of the time the Product Owner is speaking and the Scrum Master, as technical supervisor, takes only minor speaking time. This was unsatisfying, especially as we have to grade both students for their presentation. Therefore we extended the role of the Scrum Master to take the responsibility for providing documentation about the past release and give a short summary of the content during the presentation. The documentation serves as an additional way of quality control for the customer. Figure 3 summarizes the mentioned tasks for each role during a release cycle.

A very important rule the students have to follow is to stay in their role, e.g. it is strictly forbidden to participate in programming while they are Product Owner or Scrum Master. These two organizational roles tend to be the most difficult to handle, although we try to give them as much support as possible. This includes giving a practice talk previous to their customer presentation in front of their tutor for feedback. Additionally we provide a document with a rough structure for their release

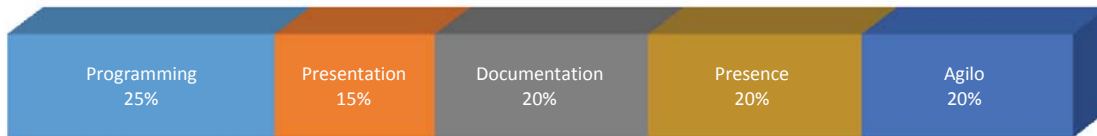


Figure 4. Assessment criteria and their weighting

documentation. It has to include a textual overview of planned requirements, charts provided by Agilo as well as a compilation of unfinished tasks and coverage reports created with the Eclipse plug-in EcEmma.

The predicted (and allowed) workload during the 16 weeks of this course is 10 hours each week for a student. One of our main goals is to provide the experience of developing a software product in a team. To achieve this we try to find a balance between the self-organized work of the student teams and some hours per week the tutor can supervise them. We demand at least two hours of meeting/working together in one of our department rooms with the tutor present because in our opinion agile software development requires a great amount of communication during the actual development. Although the teams each year differ greatly in their way of how to get the work done, most of the teams work up to or even more than 10 hours together with their team mates in some rooms of the university.

In addition to the project work, there is a weekly organizational meeting with their tutors. These meetings are mostly used to discuss problems the Scrum Master cannot handle.

After each release the Product Owner and Scrum Master of a team change. As a team starts with eight team members at the beginning of the course, each student has to undertake one of these two roles during one release.

At the end of each release statistics about the current state of the subversion repositories are generated for each team by the research assistants. This addresses two issues: At first, each developer can get an overview of his work during the past release. At second, students that will probably fail the course because of insufficient lines of code can be identified as early as possible. These students are invited for a conversation to find out, whether there is a personal or team related problem, which can be solved. Sometimes they underestimated the necessary effort, as described in the next section, to succeed in this course. Those students mostly end their participation of the course to not affect the team performance negatively for the remaining time.

After the fourth release and the last release presentation a final tournament ends the course. At this event the developed AI implementations compete against each other and the teams can present their developed software to the other students. This social event is not mandatory, but with around 60%-70% of the students participating quite popular amongst them. It is mostly used to discuss their experiences during the course with other participants or the course's staff. Additionally the tournament serves as a motivation for the last release to finish all open tasks and present a working project result.

4.3. After the Course

To grade the students as objective as possible the different criteria and their weight are published at the beginning of the course and visualized in Figure 4.

Although the students usually tend to spend most of their time in the programming part of the software development, we explicitly weight this part only for 25% and have moderate requirements

of at least 1000 lines of tested and productive code for each student. With this approach we try to achieve a more team oriented way of working. This means we highly encourage pair programming between more and less experienced students as well as a deep involvement with the roles of Product Owner and Scrum Master. As the documentation and the presentation are tasks of this roles, they affect the grade with combined 35% the most. Each presentation is graded by up to three people including the customers and an additional research assistant and feedback is provided immediately. The presence is weighted with 20%. This includes on the one hand the physical presence of a student during the two required hours a week and on the other hand his behavior during this presence time; does he communicate with his team members and is he working with or against his team. The team's tutor is responsible for documenting and reporting these incidents to the research assistants.

While the first four blocks are individual criteria, the Agilo criterion is a team wide grade. This way we again try to encourage team work as every student has to take care of the teams Agilo. In our experience during the first release the Agilo instance is maintained very poorly until the students understand how the tool has to be used and start to follow the Scrum methodology. The remaining releases tend to be much better, although there remains a big difference in the quality across the Agilo instances of all teams.

5. Evaluation and Conclusion

The focus of this paper is to give detailed insight into the course structure of SE1. However we also evaluated several aspects during and after the course in the form of anonymous paper and online surveys. After each release we evaluated the time per week students spent for this course. The first conclusion we got was that the work load of this course is very high, but acceptable. Although many students spent more than the recommended 10 hours per week for this course, a part of this time is spent voluntarily to improve the software by adding additional features apart the requirements. Second, the results show that the students tend to favor working in a team over working alone. Almost 2/3 of the participants worked at least five hours and around 20% even more than 10 hours together with their team members.

After the course we gave a more comprehensive survey to the students with a rating scale from 1 (not sufficient) to 6 (very good). The results show that approximately 80% of the students are satisfied with the content and structure of SE1. Analysis of questions concerning technical and social skill improvements reveal that skills like programming and handling of tools like Eclipse and Subversion improved as expected: These skills were rated from satisfying to very good by 70% of the students. The results for the primary objective about team work are only ordinary and not satisfying since only 24% of the students rated their team work good or very good. As we already put a lot of effort in the team building process at the beginning of the course, this result underlines how unsatisfying the work in teams with very heterogeneous characters and preexisting knowledge can be. Concerning the presentation skills, 68% of the students rated their improvement between 3 and 5 which meets our expectations. The feedback for the usage of Agilo was very negative with about 60% with a rating of 2 or lower.

Concluding, the structure of this course became well-established over the past four years and the Scrum process matches our needs for an agile software engineering course very well. As we mentioned earlier, this course is very time consuming for the staff. Nevertheless this high effort is mandatory to organize the course as realistic as possible. But with the large staff the communication tends to become problematic. As the students normally only talk to the customers or their tutors the feedback reaches the research assistants respectively the professor only indirectly. So if an unsolved problem is not reported in time to the research assistants this can affect a whole team negatively.

This problem could be tackled by collecting active and passive feedback more often. Therefore, a passive participation of one research assistant in the weekly meeting of each team seems like a possible solution. We realized this problem occurs mainly because the tutors spend a different amount of time to help their teams and some tend to wait very long whether a problem can be solved by the team themselves. In contrast, other tutors interfere fast without giving the team a chance to self-manage the problem. Without a lot of additional effort by the research assistants this behavior is very difficult to prevent.

The randomly created teams caused some complaints and negative feedback. Nonetheless we will not change it because of two reasons. On the one hand random teams allow much more realistic member constellations as the characteristics and skills of the students vary more. On the other hand we noticed that the average grades are better with random than with self-created teams. The reason is that these self-created teams are much more homogeneous. This means the more experienced programmers will form their teams with other good programmers and perform very well while the teams with mostly inexperienced programmers will probably fail the whole course.

The available (open-source) tools to support the students during their work in the Scrum process are not satisfying. Although we evaluated and used several tools during the past courses our negative experiences are confirmed by the students' evaluation. However, Agilo is still the best tool we found so far.

Based on the results of the evaluation we think the starting phase of the project work can be improved in several areas. As the newly formed teams perform very different in the beginning, especially the "weaker" teams should be supervised more carefully to find the reasons behind their slow progress. During the first release most teams have problems in the adaption of the Scrum process in combination with Agilo. Therefore a small example set of requirements, user stories and tasks should be provided by the research assistants at the beginning of the course. Those should give them support to understand the required workflow.

References

- [1] Agilo Software GmbH. Agilo for Scrum. <http://bit.ly/oz9D85>, 2012. Online; accessed 16-October-2012.
- [2] C. Coupal and K. Boechler. Introducing agile into a software development capstone project. In *Agile Conference, 2005. Proceedings, Introducing agile into a software development Capstone project*, pages 289–297, 2005.
- [3] S. Karunasekera and K. Bedse. Preparing software engineering graduates for an industry career. In *Software Engineering Education Training, 2007. CSEET '07. 20th Conference on*, pages 97–106, july 2007.
- [4] A. Kochtchi, W. Koczewski, and F. Hager. Kunagi. <http://bit.ly/9YrhU3>, 2012. Online; accessed 23-October-2012.
- [5] V. Mahnic. A capstone course on agile software development using scrum. *Education, IEEE Transactions on*, 55(1):99–106, 2012.
- [6] F. Meawad. The virtual agile enterprise: Making the most of a software engineering course. In *Proceedings of the 2011 24th IEEE-CS Conference on Software Engineering Education and Training, CSEET '11*, pages 324–332. IEEE Computer Society, 2011.
- [7] T. Reichlmayr. The agile approach in an undergraduate software engineering course project. In *Frontiers in Education, 2003. FIE 2003 33rd Annual, The agile approach in an undergraduate software engineering course project*, volume 3, pages S2C – 13–18 vol.3, 2003.
- [8] D. Rico and H. Sayani. Use of Agile Methods in Software Engineering Education. In *Agile Conference, 2009. AGILE '09*, pages 174–179, 2009.
- [9] K. Schwaber and M. Beedle. *Agile Software Development with Scrum*. Prentice Hall PTR, Upper Saddle River, NJ, USA, 1st edition, 2001.
- [10] VersionOne. State of agile development. <http://bit.ly/12NVx6U>, 2012. Online; accessed 10-April-2013.